TLM Creation Overview

Rapid Training

W-2024.09

Labs - Overview

© 2024 Synopsys, Inc.

Lab Exercises - Introduction

This training program aims at teaching you the basics as well as some advanced capabilities of the TLM Creation perspective of Virtualizer Studio. For more information on the model creation methodology, please refer to the *TLM Creator User's Guide* as well as the *SystemC Modeling Library Reference Manual* from the Virtual Prototyping documentation.

The training labs are designed to be interleaved with the training lectures. The tool and methodology specific knowledge required to complete the labs are covered in the respective lecture (or prior lectures). This interleaved *learning & doing* results in a deeper and faster understanding of the underlying techniques and methodology.

The TLMC training sessions as well as the labs are designed in a modular way. All labs either start from scratch or come with a starting point (in the `start_ModuleXLabY` directory). This allows you to skip modules or labs if they cover known topics or topics that are not required by you.

The starting point data also serves as sample solution for the previous lab exercise. For example, the starting point data `start_Module2LabC` is the TLMC sample solution for Module 2 Lab B. For the sample solution of the Module1LabD, refer to the `timer` model sources under the `module_1/start_Moudle1LabG` folder as that's the logical subsequent lab – the intermittent Labs E and F may be added at a later point.

Note: This Training material is compatible with Virtualizer or Platform Architect Version V-2024.06. Please contact `Synopsys support` in case of any issues.

Lab Exercises Available in the Training Course

This training program includes the following lab exercises and documents:

Lab Exercises Available in the Training Course **1**

Module 1 – Building an Interrupt Timer model **2**

- Module 1 Lab A: Create the Model Skeleton of an Interrupt Timer 2
- Module 1 Lab B: Test Driven Design with TLMC 2
- Module 1 Lab C: Implement the Behavior of an Interrupt Timer 2
- Module 1 Lab D: Model Debugging 2
- *Module 1 Lab E: Line Coverage (Not available yet)* 2
- *Module 1 Lab F: Advanced Functional Coverage Features (Not available yet)* 2
- Module 1 Lab G: Model Usage in a VDK 3

Module 2 – Building a SPI Controller Model **3**

- Module 2 Lab A: Create the Model Skeleton of an SPI Controller 3
- Module 2 Lab B: Implement the Behavior of an SPI Controller 3
- Module 2 Lab C: Add Versioning Support to the SPI Controller 3
- Module 2 Lab D: Test and Coverage for the SPI Controller Model 3

Module 3 – Import Existing SystemC TLM Models **3**

- Module 3 Lab A: Import Existing SystemC TLM Models 3

Training Specifications **4**

Lab-Data Package Contents **4**

Module 1 – Building an Interrupt Timer model

This module covers steps to create and build on an Interrupt Timer model using the TLM Creation technology of Virtualizer Studio. The module includes the following lab exercises.

- ☐ Module 1 Lab A: Create the Model Skeleton of an Interrupt Timer
  In this lab, you will create the shell of the PIT timer model using an empty template in the TLM Creation perspective of Virtualizer Studio as a starting point. For the specification of the memory elements (register files, register, bit-fields), you can optionally use an IP-XACT file to import these definitions into the TLMC project.

  This introductory lab will also take you through the infrastructure of tool such as the context specific help system, the generated directories and files for a TLMC project, the basics of the unit-testing environment.

- ☐ Module 1 Lab B: Test Driven Design with TLMC
  Your task for this lab is to use the TLM Creation perspective of Virtualizer Studio to test and implement some additional behavior for the timer model that you created in Module 1 Lab A.

  The model creation flow starts with the Design Model phase, wherein you review the requirement specs and decide on the model architecture, create a Model Feature Tracker, listing the desired features/functionalities.

  Next is the *Create Test Plan* phase, where you create a test plan, and add unit tests for the listed features. Moving on to *Add Behavior* phase, where you add the functional behavior code for the model.

  Once behavior is added, the *Testing and Debugging* phase begins, wherein the component test suite is to be built and run on the model implementation. You analyze the test results. All steps from model design to testing, and debugging are to be executed iteratively, till all the model features are implemented and tested.

- ☐ Module 1 Lab C: Implement the Behavior of an Interrupt Timer
  Your task for this lab is to the implement the `set-and-forget` behavior of the timer as described in the specification of the interval timer and as listed in the sample feature tracking spreadsheet, `TimerFunctionalityTracking.xlsx`.

  During this lab, you will see how to create TLM models in an efficient way, using TLM Creator's productivity features such as callbacks, continuous assignments, a library of re-use objects, etc.

- ☐ Module 1 Lab D: Model Debugging
  Your task for this lab is to implement one additional aspect of the timer behavior, debug the code and fix the issues. For debugging we will use multiple techniques, starting from using built-in FastTrack messages in callbacks and reuse objects, using a source code debugger to step through and analyze the code, as well as using VDK Debug features to inspect the value of registers or your model.

- ☐ *Module 1 Lab E: Line Coverage (Not available yet)*

- ☐ *Module 1 Lab F: Advanced Functional Coverage Features (Not available yet)*

- [ ] Module 1 Lab G: Model Usage in a VDK
  You can integrate models developed or imported in the TLM Creation perspective easily into your virtual prototype design using the VDK Creation perspective of Virtualizer Studio. Your task for the current exercise to go through the steps to add the timer model (created in the earlier labs) in a VDK project in Virtualizer Studio or Platform Creator tool.

Module 2 – Building a SPI Controller Model

This module covers steps to create and build on a SPI Controller model using the TLM Creation technology of Virtualizer Studio. The module includes the following lab exercises.

- [ ] Module 2 Lab A: Create the Model Skeleton of an SPI Controller
  Your task for this module is to create and test a model of a Serial Protocol Interface (SPI) controller peripheral using the TLM Creation perspective of VS, that comes with a broad set of productivity features, including model versioning, continuous assignments, reuse objects, or protocol engines.

  Please refer to the SPI Controller Specification for interface and behavior of this training example model.

  In this first lab, you will create the skeleton of the SPI controller, i.e. create a new project for the new model, add parameters for behavior and versioning, as well as interfaces and a memory map.

- [ ] Module 2 Lab B: Implement the Behavior of an SPI Controller
  In the previous Lab, you've been creating the shell of a Serial Protocol Interface (SPI) controller peripheral – the task for this lab is to implement the behavior of the model. Using the library of re-use objects, you can keep the amount of code you have to write and test at a minimum, and features such as continuous assignments also help to keep the model implementation concise and efficient.

- [ ] Module 2 Lab C: Add Versioning Support to the SPI Controller
  In the module 2 Lab B, you've been implementing the base functionality of the Serial Protocol Interface (SPI) controller peripheral – the task for this lab is to implement a new version with additional features of the model, and to add versioning information to the project to be able to maintain multiple versions of the same model in one TLMC project. This makes it easier to maintain and develop the model over time.

- [ ] Module 2 Lab D: Test and Coverage for the SPI Controller Model
  In the previous labs, Lab C and Lab D you've been implementing the base and incremental versioned functionality of the Serial Protocol Interface (SPI) controller peripheral – the task for this lab is to test the model using the Unit Testing framework, including measuring the functional coverage.

Module 3 – Import Existing SystemC TLM Models

This module covers steps to *Import Existing SystemC TLM Models in* Virtualizer Studio. The module includes the following lab exercises.

- [ ] Module 3 Lab A: Import Existing SystemC TLM Models
  Your task for this lab is to use TLM Creator to import an existing SystemC TLM Model. This flow is typically used when you already have SystemC models or testbench parts (either written manually or as a product of other flows) that shall be used in a VDK.

  You will start with the SystemC source files provided in the lab data package and import it in your TLMC project. The tool extracts the model properties from the C++ source code. Then, you will build the project so that the model can be used for VDK creation.

Training Specifications

- Training Programmable Interval Timer Specification
  A specification of a programmable interval timer model for the training labs of Module 1.

- Training SPI Controller Specification

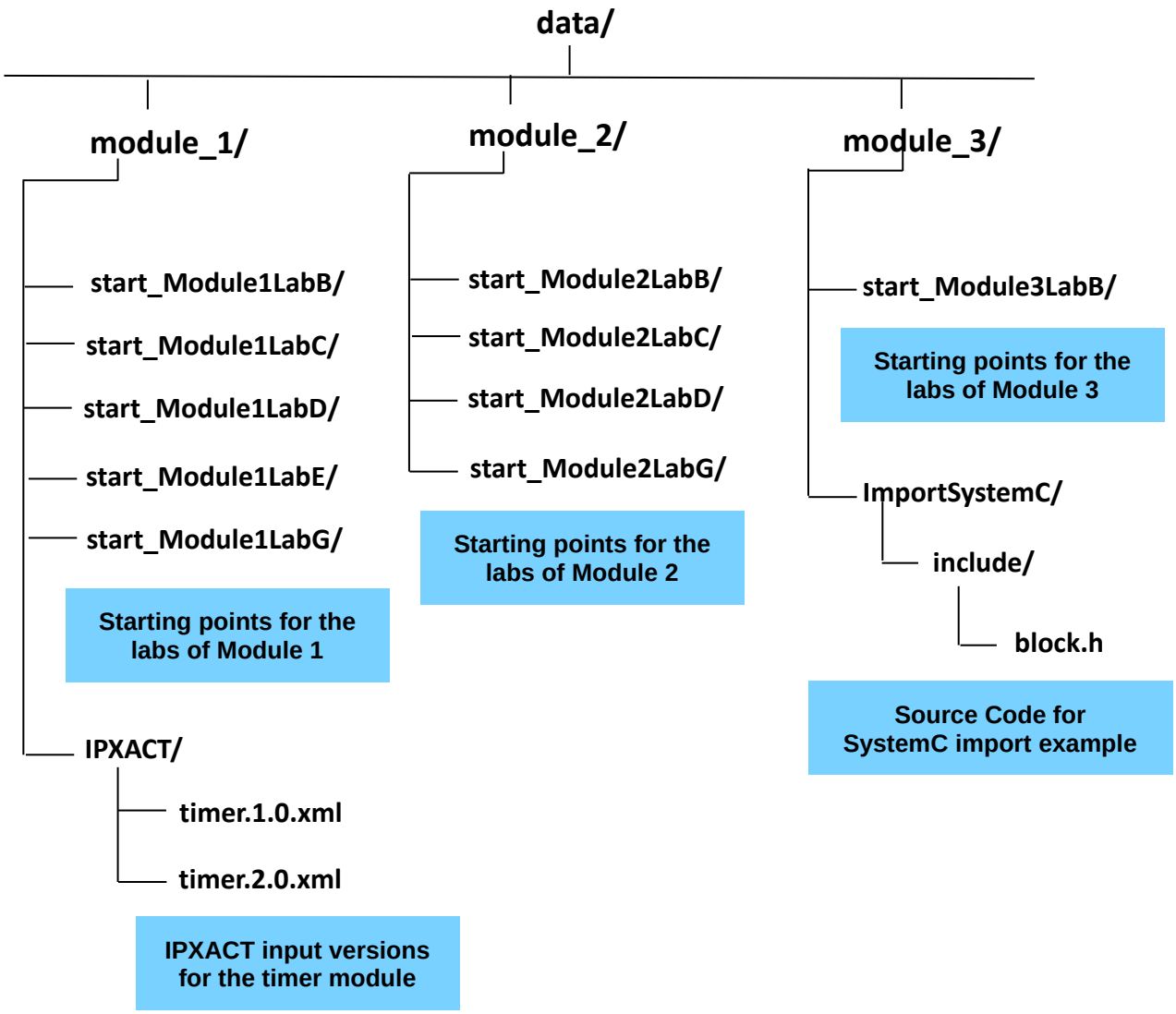
A specification of a simplified Serial Peripheral Interface (SPI) Controller model for the training labs of Module 2.

Lab-Data Package Contents

Before starting with the labs, untar the lab-data contents from the **tlmcTrainingLabdata.tar.gz** package.

The TLMC training sessions as well as the labs are designed in a modular way. All labs either start from scratch or come with a starting point (in the `start_ModuleXLabY` directory). This allows you to skip modules or labs if they cover known topics or topics that are not required by you.

The starting point data also serves as sample solution for the previous lab exercise. For example, the starting point data `start_Module2LabC` is the TLMC sample solution for Module 2 Lab B.

For the sample solution of the Module1 Lab D, refer to the `timer` model sources under the `module_1/start_Moudle1LabG` folder as that's the logical subsequent lab.

Let's explore the contents of the Lab-data (***tlmcTrainingLabdata.tar.gz***) attached along with the training package for clarity.